AULA 6.5

Métricas implementadas na mão

Matriz de confusão, precision, recall, F1, ROC e AUC construídas a partir de contagens, antes de qualquer chamada a sklearn.metrics.

2 horas

Prática: classification_report do zero



Ao final da aula, você vai conseguir

01 Montar a matriz de confusão a partir de y_real e y_pred e nomear TP, TN, FP e FN.

02 Calcular accuracy, precision, recall e F1 à mão e explicar o que cada uma mede.

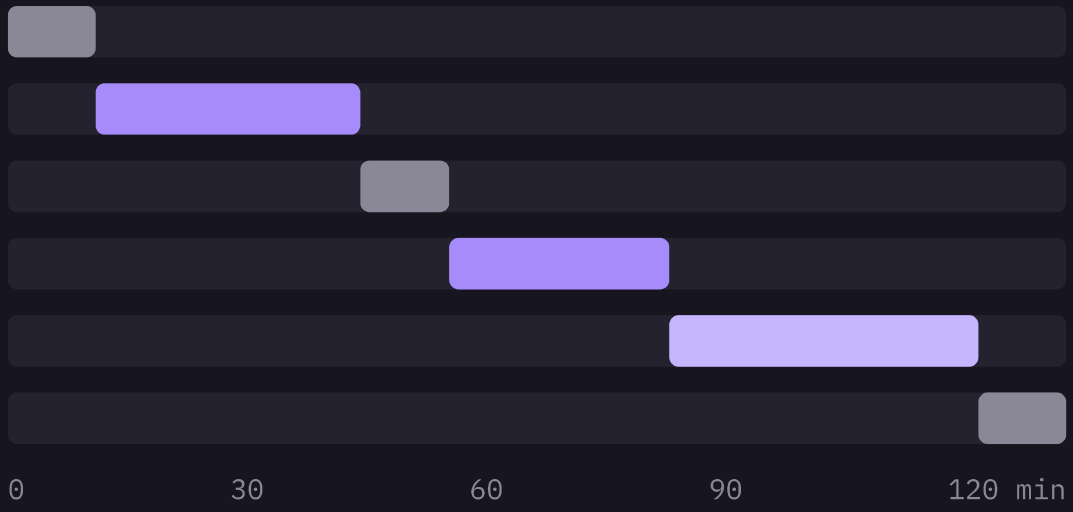
03 Escolher a métrica adequada a partir do custo de cada tipo de erro.

04 Construir a curva ROC variando o limiar e calcular a AUC pela regra dos trapézios.

05 Escrever o seu próprio classification_report em NumPy e conferir com o scikit-learn.

Roteiro das duas horas

0–10′	Objetivos e recapitulação
10–40′	Matriz de confusão e métricas
40–50′	Exemplo resolvido e quiz
50–75′	Relatório, ROC e AUC
75–110′	Exercício guiado em NumPy
110–120′	Erros comuns, desafio e leitura



Regressão logística: das características à classe

1 · ENTRADA

x

características: nº de links, palavra "grátis"…

2 · SCORE

$z = w \cdot x + b$

combinação linear, como na regressão linear

3 · SIGMOIDE

$\sigma(z) = 1/(1+e^{-z})$

espreme z para o intervalo (0, 1)

4 · PROBABILIDADE

$p = P(y=1|x)$

chance estimada de ser da classe 1

5 · LIMIAR

$p \geq 0{,}5 \rightarrow 1$

abaixo do limiar, prevê 0

`classe 1` positiva, a que queremos detectar (spam). classe 0: negativa (mensagem normal).

`treino` gradiente descendente minimiza a entropia cruzada e ajusta w e b.

Como saber se o modelo classifica bem?

90 / 100

previsões corretas. Esse modelo é bom?

DEPENDE 1

Quais foram os 10 erros?

Mandar um doente para casa custa mais do que pedir um exame extra a alguém saudável.

DEPENDE 2

Quantos exemplos há em cada classe?

Se 90 das 100 mensagens já são normais, responder “normal” sempre também acerta 90.

Acurácia: a fração de previsões corretas

Um filtro avaliou 100 mensagens. Com que frequência ele acertou?

$$\text{Accuracy} = \frac{\text{acertos}}{\text{total de previsões}}$$

acertos previsão igual ao valor real (0 ou 1)

total número de exemplos avaliados

`90 / 100 = 0,90` 90% de acerto

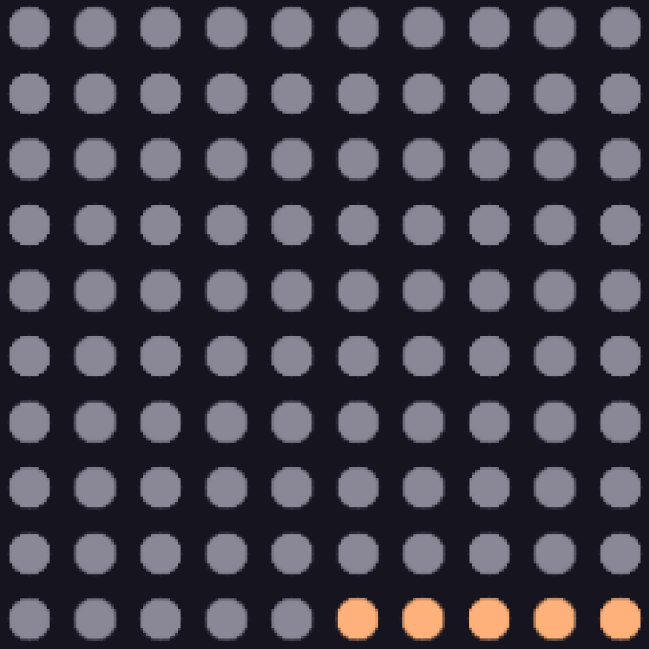
```python
import numpy as np

y_real = np.array([1, 0, 0, 1, 0])
y_pred = np.array([1, 0, 1, 1, 0])

acertos = (y_real == y_pred)
# [True, True, False, True, True]

accuracy = acertos.mean()
# True vale 1, False vale 0
# 4 / 5 = 0.8
```

Quando a acurácia engana: classes desbalanceadas



95 normais 5 spams

MODELO

Prevê “normal” para todas

Accuracy

95%

95 / 100

Spams detectados

0

de 5

A classe majoritária domina a conta. Compare sempre com a linha de base de prever a classe mais frequente.

Matriz de confusão: classe verdadeira × classe prevista



CONVENÇÃO DA AULA

Linhas = classe verdadeira. Colunas = classe prevista. Classe 0 primeiro, como no scikit-learn.

COMO LER O NOME

Positive/Negative é o que o modelo disse. **True/False** diz se ele acertou.

DIAGONAL

TN e TP são acertos. FP e FN, fora da diagonal, são erros.

Os quatro resultados no filtro de spam

Classe positiva (1) = spam. Classe negativa (0) = mensagem normal.

TN real 0 · previsto 0

Mensagem normal entregue na caixa de entrada.

Acerto: nada a fazer.

FP real 0 · previsto 1

A proposta de emprego foi parar no spam.

Alarme falso: o usuário perde algo importante.

FN real 1 · previsto 0

Um golpe passou e chegou à caixa de entrada.

Escapou: o spam não foi detectado.

TP real 1 · previsto 1

Spam identificado e bloqueado.

Acerto: o filtro fez o seu trabalho.

Construindo a matriz com dez previsões

exemplo	1	2	3	4	5	6	7	8	9	10
y_real	1	0	1	1	0	0	1	0	0	1
y_pred	1	0	0	1	1	0	1	0	0	1
resultado	TP	TN	FN	TP	FP	TN	TP	TN	TN	TP

	prev. 0	prev. 1
real 0	4 TN	1 FP
real 1	1 FN	4 TP

Confira: 4 + 1 + 1 + 4 = 10 previsões. A diagonal guarda os acertos.

$$\text{Accuracy} = \frac{TP + TN}{\text{total}} = \frac{4 + 4}{10} = 0{,}80$$

Quiz rápido: a matriz de confusão

1

O que é um verdadeiro positivo?

2

Qual a diferença entre falso positivo e falso negativo?

3

Um modelo com 95% de acurácia é necessariamente bom?

4

Um exame acusa câncer em uma pessoa saudável. FP ou FN?

Precision: quanto confiar em um “positivo”

Entre tudo que o modelo classificou como positivo, quantas previsões estavam corretas?

AS 10 MENSAGENS QUE O MODELO MARCOU COMO SPAM

TP	TP	TP	FP	TP	TP	TP	TP	FP	TP

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{8}{8 + 2} = 0{,}80$$

TP disse spam e era spam

FP disse spam, mas era normal

TP+FP tudo que o modelo chamou de spam

Precision 0,80: interpretação e quando usar

USA SÓ A COLUNA “PREVISTO 1”

TN	FP
FN	TP

Accuracy usa as quatro células. Precision não olha para TN nem FN.

O QUE 0,80 SIGNIFICA

De cada 10 “positivos” do modelo, 8 estão certos e 2 são alarmes falsos.

IMPORTA QUANDO O FP É CARO

E-mail de cliente enviado para a lixeira

Cartão bloqueado em compra legítima

Conta congelada por suspeita de fraude

```
tp, fp = 8, 2

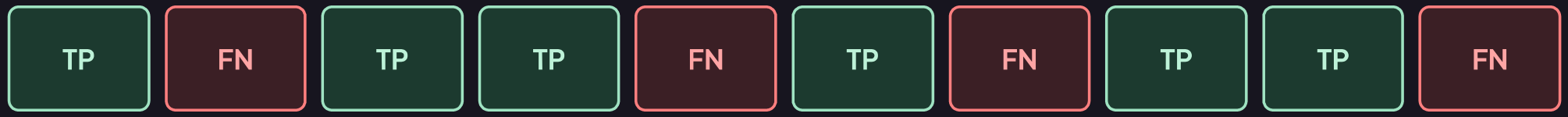
def precision(tp, fp):
    if tp + fp == 0:
        # o modelo não previu
        # nenhum positivo
        return 0.0
    return tp / (tp + fp)

precision(tp, fp)  # 0.8
```

Recall: quantos positivos reais o modelo encontra

Entre todos os exemplos realmente positivos, quantos o modelo conseguiu identificar?

OS 10 PACIENTES QUE REALMENTE TÊM A DOENÇA



$$\text{Recall} = \frac{TP}{TP + FN} = \frac{6}{6 + 4} = 0{,}60$$

TP era doente e o exame detectou

FN era doente, mas o exame não viu

TP+FN todos os positivos que existem

Recall 0,60: interpretação e quando usar

USA SÓ A LINHA “REAL 1”

TN	FP
FN	TP

Também chamada de sensibilidade ou TPR (True Positive Rate).

O QUE 0,60 SIGNIFICA

O modelo encontrou 60% dos positivos reais. 4 em cada 10 escaparam.

IMPORTA QUANDO O FN É CARO

Paciente com câncer mandado para casa

Fraude que passa sem alerta

Peça defeituosa aprovada na inspeção

```
tp, fn = 6, 4

def recall(tp, fn):
    if tp + fn == 0:
        # não existe nenhum
        # positivo real
        return 0.0
    return tp / (tp + fn)

recall(tp, fn)  # 0.6
```

Precision vs recall: dois modelos, mesmos dados

100 mensagens: 20 spams e 80 normais.

Modelo A conservador

TN 79	FP 1
FN 12	TP 8

Precision · 8/9 **0,89**

Recall · 8/20 **0,40**

Quase todo alerta é verdadeiro, mas 12 spams passam.

Modelo B agressivo

TN 58	FP 22
FN 2	TP 18

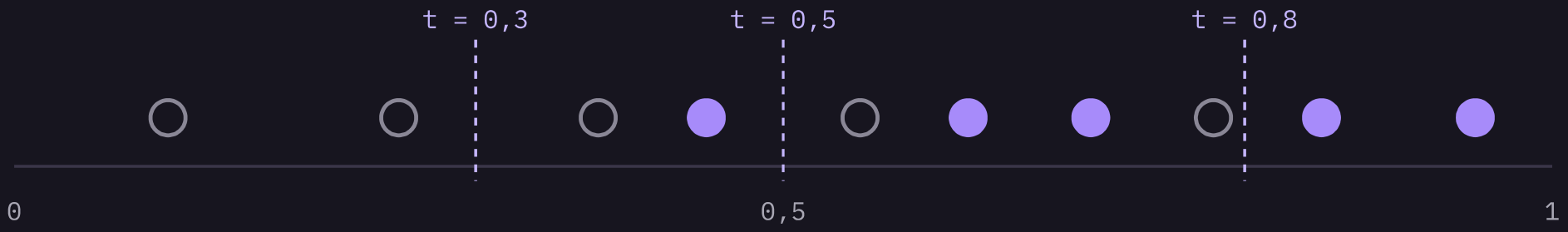
Precision · 18/40 **0,45**

Recall · 18/20 **0,90**

Quase nenhum spam passa, mas 22 e-mails bons são bloqueados.

O limiar muda o equilíbrio entre precision e recall

Regra: prever 1 se p ≥ limiar. Dez mensagens, cinco spams reais.



spam real (1) normal real (0) eixo: probabilidade prevista p

limiar	TP	FP	FN	TN	precision	recall
0,8	2	0	3	5	1,00	0,40
0,5	4	2	1	3	0,67	0,80
0,3	5	3	0	2	0,625	1,00

F1-score: precision e recall em um só número

Queremos um número que só seja alto quando as duas forem altas.

$$F1 = 2 \times \frac{P \times R}{P + R}$$

P, R	precision e recall, entre 0 e 1
P × R	fica pequeno se qualquer um for pequeno
2 ×	faz F1 = P quando P = R

Mesma conta com contagens:
`F1 = 2TP / (2TP + FP + FN)`

EXEMPLO · P = 0,80 · R = 0,50

```
2 × 0,80 × 0,50 = 0,80

0,80 + 0,50 = 1,30

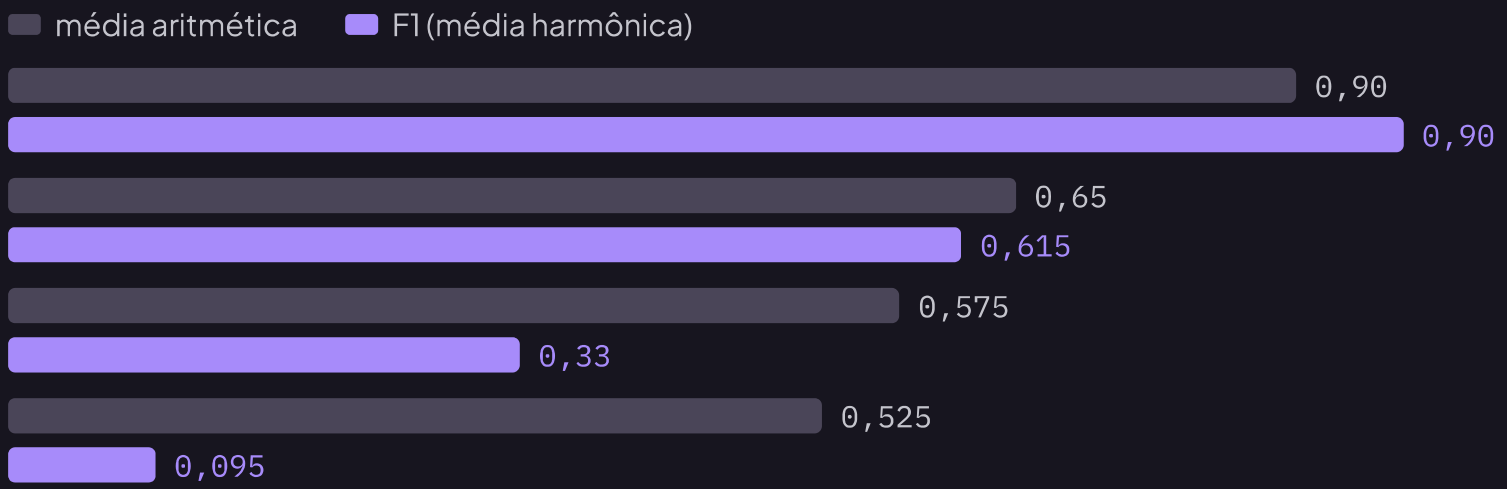
F1 = 0,80 / 1,30 ≈ 0,6154
```

`média aritmética = 0,65`

O F1 fica mais perto do menor valor: é a média harmônica.

Por que média harmônica e não aritmética

P	R
0,90	0,90
0,80	0,50
0,95	0,20
1,00	0,05



A média harmônica é dominada pelo menor valor. Limite: o F1 dá o mesmo peso a FP e FN e ignora TN.

Exemplo completo, resolvido à mão

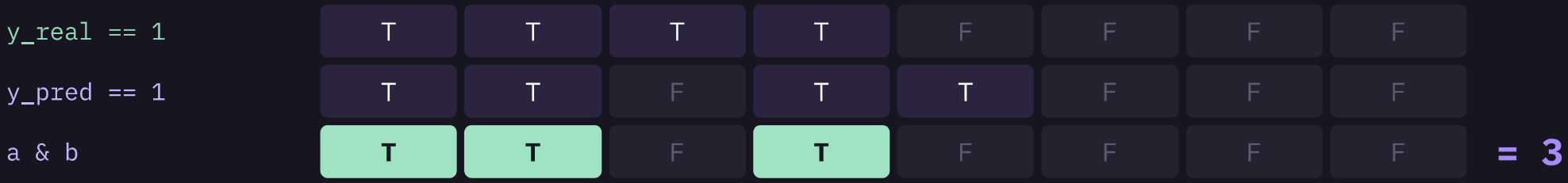
y_real	1	1	1	1	0	0	0	0
y_pred	1	1	0	1	1	0	0	0
resultado	TP	TP	FN	TP	FP	TN	TN	TN

	prev. 0	prev. 1
real 0	3 TN	1 FP
real 1	1 FN	3 TP

métrica	conta	valor
Accuracy	(3 + 3) / 8	0,75
Precision (1)	3 / (3 + 1)	0,75
Recall (1)	3 / (3 + 1)	0,75
F1 (1)	2·0,75·0,75 / 1,50	0,75

Contando TP com comparações booleanas

```
np.sum((y_real == 1) & (y_pred == 1))
```



==

compara cada elemento e devolve um array de True/False

&

“E” lógico posição a posição. O and do Python não funciona com arrays

( )

& tem precedência maior que ==. Sem parênteses a conta muda

np.sum

True vale 1 e False vale 0: a soma conta os True

As métricas binárias em NumPy, do zero

```python
def contar(y_real, y_pred):
    tp = np.sum((y_real == 1) & (y_pred == 1))
    tn = np.sum((y_real == 0) & (y_pred == 0))
    fp = np.sum((y_real == 0) & (y_pred == 1))
    fn = np.sum((y_real == 1) & (y_pred == 0))
    return tp, tn, fp, fn

def matriz_confusao(y_real, y_pred):
    tp, tn, fp, fn = contar(y_real, y_pred)
    # linhas = real, colunas = prevista
    return np.array([[tn, fp],
                     [fn, tp]])

y_real = np.array([1, 1, 1, 1, 0, 0, 0, 0])
y_pred = np.array([1, 1, 0, 1, 1, 0, 0, 0])
tp, tn, fp, fn = contar(y_real, y_pred)
# 3 3 1 1  ->  [[3, 1], [1, 3]]
```

```python
def divide(num, den):
    # den == 0: métrica indefinida
    # convenção: devolve 0.0
    return num / den if den != 0 else 0.0

def accuracy(tp, tn, fp, fn):
    return divide(tp + tn, tp + tn + fp + fn)

def precision(tp, fp):
    return divide(tp, tp + fp)

def recall(tp, fn):
    return divide(tp, tp + fn)

def f1_score(p, r):
    return divide(2 * p * r, p + r)

# todas devolvem 0.75 neste exemplo
```

Quiz rápido: precision, recall e F1

1

O que precision mede?

2

O que recall mede?

3

Por que utilizamos o F1?

4

Falsos negativos são muito caros. Qual métrica vigiar?

5 TP = 12, FP = 3, FN = 8. Calcule precision, recall e F1.

Três classes: matriz 3×3 e one-vs-rest

25 pinguins. Para cada classe, ela vira “positiva” e as outras viram “negativas”.

real ↓ prev →	Adelie	**Chinstrap**	Gentoo
Adelie	8 TN	2 FP	0 TN
Chinstrap	1 FN	3 TP	1 FN
Gentoo	0 TN	1 FP	9 TN

CHINSTRAP COMO POSITIVA

TP diagonal: 3

FN resto da linha: 1 + 1 = 2

FP resto da coluna: 2 + 1 = 3

TN todo o resto: 25 − 8 = 17

precision = 3 / 6 = 0,50

recall = 3 / 5 = 0,60

F1 = 6 / 11 ≈ 0,545

Métricas por classe e três formas de média

classe	TP	FP	FN	precision	recall	f1	support
Adelie	8	1	2	0,889	0,800	0,842	10
Chinstrap	3	3	2	0,500	0,600	0,545	5
Gentoo	9	1	1	0,900	0,900	0,900	10
macro				0,763	0,767	0,763	25
weighted				0,816	0,800	0,806	25
micro	20	5	5	0,800	0,800	0,800	25

Macro
média simples: cada classe pesa igual

Weighted
média ponderada pelo support (amostras reais)

Micro
soma TP, FP e FN antes de dividir. Aqui = accuracy

Anatomia do classification_report

	precision	recall	f1-score	support
Adelie	0.89	0.80	0.84	10
Chinstrap	0.50	0.60	0.55	5
Gentoo	0.90	0.90	0.90	10
accuracy			0.80	25
macro avg	0.76	0.77	0.76	25
weighted avg	0.82	0.80	0.81	25

precision · recall · f1-score
one-vs-rest de cada classe

support
amostras reais da classe (soma da linha), não previsões

accuracy
acertos / total; aparece na coluna f1-score por convenção

macro avg
média simples das classes

weighted avg
média ponderada pelo support

Construindo o seu relatório em oito passos

1. Identificar as classes
2. Construir a matriz k×k
3. Métricas por classe
4. Support de cada classe
5. Macro average
6. Weighted average
7. Accuracy
8. Formatar a tabela

Denominador zero → 0.0, como zero_division=0. Documente a escolha.

```
# 1
classes = np.unique(np.concatenate([y_real, y_pred]))
# 2  M[i, j] = reais da classe i previstos como j
# 3  one-vs-rest vetorizado
tp = np.diag(M)
fp = M.sum(axis=0) - tp   # coluna sem a diagonal
fn = M.sum(axis=1) - tp   # linha sem a diagonal
precision = razao(tp, tp + fp)
recall    = razao(tp, tp + fn)
f1        = razao(2 * precision * recall,
                  precision + recall)
# 4
support = M.sum(axis=1)
# 5, 6, 7
macro_p    = precision.mean()
weighted_p = np.average(precision, weights=support)
acc        = tp.sum() / M.sum()
```

ROC: TPR e FPR para cada limiar

$$\text{TPR} = \frac{TP}{TP + FN}$$

fração dos positivos reais detectados. É o recall.

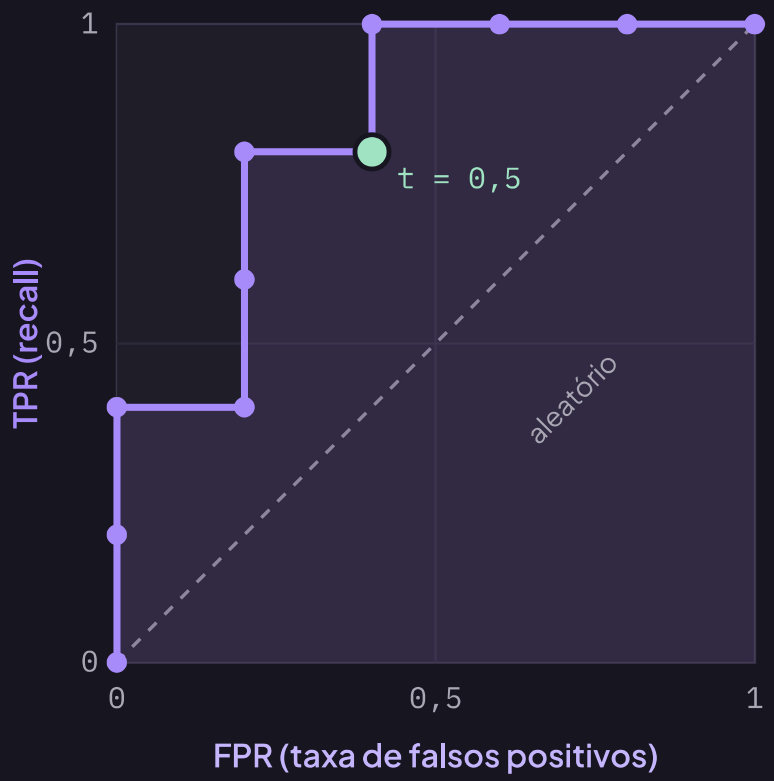
$$\text{FPR} = \frac{FP}{FP + TN}$$

fração dos negativos reais classificados como positivos.

Regra: prever 1 se p ≥ limiar. 5 positivos e 5 negativos.

limiar	TP	FP	TN	FN	TPR	FPR
∞	0	0	5	5	0,0	0,0
0,95	1	0	5	4	0,2	0,0
0,85	2	0	5	3	0,4	0,0
0,78	2	1	4	3	0,4	0,2
0,70	3	1	4	2	0,6	0,2
0,62	4	1	4	1	0,8	0,2
0,55	4	2	3	1	0,8	0,4
0,45	5	2	3	0	1,0	0,4
0,38	5	3	2	0	1,0	0,6
0,25	5	4	1	0	1,0	0,8
0,10	5	5	0	0	1,0	1,0

A curva ROC: FPR no eixo X, TPR no eixo Y



`(0, 0)`

limiar infinito: nunca prevê positivo

`(1, 1)`

limiar mínimo: tudo vira positivo

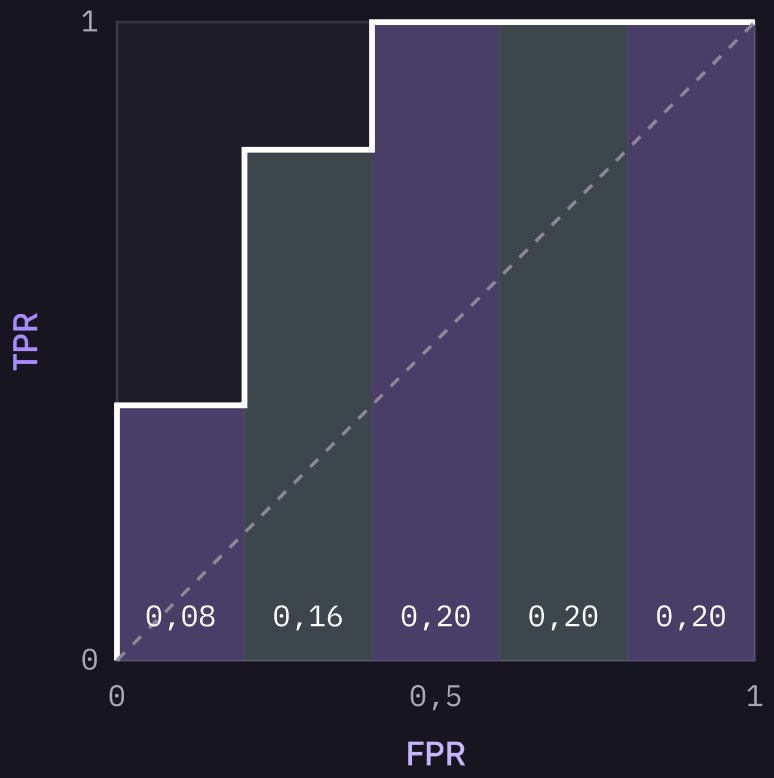
`diagonal`

score sem capacidade de separar as classes

`por que scores?`

classes de um único limiar dão um só ponto. A curva exige varrer o limiar.

AUC: a área sob a curva pela regra dos trapézios



$$A = (x_2 - x_1) \times \frac{y_1 + y_2}{2}$$

$x_2 - x_1$ largura: avanço em FPR

$(y_1 + y_2) / 2$ altura média: TPR nos dois pontos

```
0,08 + 0,16 + 0,20 + 0,20 + 0,20
AUC = 0,84
```

Mesma resposta por ordenação: em 21 de 25 pares (positivo, negativo), o positivo tem score maior. 21/25 = 0,84.

Interpretando a AUC e implementando do zero



≈ 1

positivos quase sempre acima dos negativos

≈ 0,5

ordena como uma moeda: sem discriminação

abaixo de 0,5

ordena ao contrário; verifique rótulos e coluna de probabilidade

AUC não é acurácia: não depende de limiar.

```python
def curva_roc(y_real, scores):
    ordem = np.argsort(-scores, kind="mergesort")
    y, s = y_real[ordem], scores[ordem]
    # empates: um corte só no fim de cada
    # grupo de scores iguais
    fim = np.r_[np.where(np.diff(s) != 0)[0],
                len(s) - 1]
    tp = np.cumsum(y)[fim]
    fp = (fim + 1) - tp
    P, N = y.sum(), len(y) - y.sum()
    tpr = np.r_[0, tp] / P   # (0,0) = limiar ∞
    fpr = np.r_[0, fp] / N
    return fpr, tpr

def auc(fpr, tpr):
    larg = fpr[1:] - fpr[:-1]
    alt = (tpr[1:] + tpr[:-1]) / 2
    return np.sum(larg * alt)      # 0.84
```

Quiz rápido: relatório, ROC e AUC

1

O que significa support?

2

Qual a diferença entre macro e weighted average?

3

O que representam os eixos da curva ROC?

4

AUC e accuracy são a mesma coisa?

5 Por que precisamos de probabilidades ou scores para desenhar uma ROC completa?

Comparando todas as métricas

métrica	o que mede	cálculo	valor alto	útil quando	limitação
Accuracy	proporção geral de acertos	`(TP+TN)/total`	acerta a maioria dos casos	classes equilibradas e erros de custo parecido	engana com classes desbalanceadas
Precision	confiabilidade dos positivos previstos	`TP/(TP+FP)`	poucos alarmes falsos	FP é caro: spam, bloqueio de cartão	ignora FN: pode deixar escapar muitos positivos
Recall	cobertura dos positivos reais	`TP/(TP+FN)`	poucos positivos escapam	FN é caro: câncer, fraude	ignora FP: prever tudo positivo dá 1,0
F1-score	equilíbrio entre precision e recall	`2PR/(P+R)`	as duas altas ao mesmo tempo	positivo raro e os dois erros importam	pesa FP e FN igual e ignora TN
ROC AUC	ordenar positivos acima de negativos	`área sob a ROC`	separa bem em vários limiares	comparar modelos antes de fixar o limiar	não mostra o desempenho no limiar usado; exige scores

Qual modelo usar na triagem de câncer?

1000 exames, 50 pacientes doentes. Discuta em dupla antes de olhar as métricas.

Modelo X

TN **943**	FP **7**
FN **22**	TP **28**

acc	prec	rec	F1
0,971	0,80	0,56	0,659

Modelo Y

TN **894**	FP **56**
FN **4**	TP **46**

acc	prec	rec	F1
0,940	0,45	0,92	0,605

X vence em accuracy e F1. Qual erro custa mais aqui: 22 doentes sem diagnóstico ou 56 exames extras?

Exercício guiado: as métricas em dez etapas

Notebook 01_exercicio_guiado.ipynb · fórmula no quadro antes de cada trecho de código

01	Carregar y_real e y_pred	2′
02	Acertos e accuracy · preveja	3′
03	Contar TP, TN, FP e FN	4′
04	Construir a matriz de confusão	3′
05	Implementar precision · preveja	2′
06	Implementar recall	2′
07	Implementar F1-score · preveja	2′
08	Estender para três classes	5′
09	Montar o relatório de classificação	7′
10	ROC e AUC do exemplo binário · preveja	5′

`preveja` escreva o resultado esperado no caderno antes de executar a célula

Erros comuns: contagens e fórmulas

erro	como corrigir
Confundir FP com FN	a 2ª palavra é o que o modelo disse: FP = disse positivo e errou
Confundir precision com recall	denominador = o que o modelo disse (precision) ou o que existe (recall)?
Accuracy como garantia	compare com a linha de base da classe majoritária (95% no spam)
F1 como média aritmética	(0,8 + 0,5)/2 = 0,65 está errado; 2PR/(P+R) = 0,615
Ignorar divisão por zero	TP+FP = 0 gera erro ou nan; trate e documente a convenção
Inverter linhas e colunas	sempre (y_real, y_pred): trocar a ordem troca FP com FN em silêncio
Não definir a classe positiva	com “sim”/“não”, declare pos_label antes de calcular

Erros comuns: relatórios, ROC e comparações

erro	como corrigir
Confundir macro e weighted	macro: classes pesam igual; weighted: pesa pelo support
Support = nº de previsões	support é a soma da linha: M.sum(axis=1), não axis=0
ROC com rótulos binários	use predict_proba(X)[:, 1], não predict(X)
Confundir AUC com accuracy	no exemplo: AUC 0,84 e accuracy 0,70 com limiar 0,5
Uma métrica só, base desbalanceada	mostre matriz, precision e recall da classe rara
Relatórios com classes diferentes	fixe labels=classes nas duas avaliações
zero_division diferentes	use a mesma política (0) no seu código e no sklearn

Desafio: reproduzir o classification_report do zero

`meu_classification_report(y_real, y_pred)` + extensão: curva_roc() e auc() com probabilidades

01 Executa sem erros

02 Matriz de confusão correta

03 Métricas calculadas corretamente

04 Apresenta as classes existentes

05 Support correto

06 Médias macro e weighted corretas

07 Relatório legível

08 Coincide com o sklearn nas mesmas configurações

09 Nenhuma função pronta de métrica nos cálculos

10 Você explica cada métrica implementada

notebook 02_desafio_classification_report.ipynb · correção no início da Aula 6.6

Leitura complementar: documentação oficial

NumPy	numpy.org/doc/stable
Guia de métricas de classificação	scikit-learn.org/stable/modules/model_evaluation.html
Confusion matrix	…/sklearn.metrics.confusion_matrix.html
Classification report	…/sklearn.metrics.classification_report.html
Precision, recall e F1	…/sklearn.metrics.precision_recall_fscore_support.html
ROC curve	…/sklearn.metrics.roc_curve.html
ROC AUC	…/sklearn.metrics.roc_auc_score.html

“…” = scikit-learn.org/stable/modules/generated